

Gruppenprojekt

Microservices-
architektur

Introduktion & Mål

KOPPLING TILL KURSMÅL

Detta moment examinerar av kursens praktiska moment. Omfattar mål: **3, 4, 5, 6, 7, 8, 9, 10, 11.**

UPPGIFTSBESKRIVNING

Ni ska i agila team bygga ett komplett, distribuerat system (t.ex. ett **bokningssystem** eller utökat Library API, eller movie-app) baserat på en microservices-arkitektur. Arbetet ska bedrivas "**API-first**" och inkludera kontinuerlig integration (CI).



1. Arkitektur & Kommunikation

FRISTÅENDE TJÄNSTER

Bygg minst två fristående microservices (t.ex. User-Service och Booking-Service).

INFRASTRUKTUR

Sätt upp en **Service Registry** och en **API Gateway** för att dirigera trafiken.

SÄKER KOMMUNIKATION

Tjänsterna ska kunna kommunicera säkert med varandra. För detta ska ni använda exempelvis **OpenFeign**.

2. Säkerhet & Skydd

CENTRALISERAD AUTENTISERING

Implementera centraliserad autentisering via er API Gateway. All inloggning ska hanteras här och resultera i en **JWT (JSON Web Token)**.

ÅTKOMSTKONTROLL I MICROSERVICES

De bakomliggande tjänsterna (t.ex. Booking-Service) ska validera token.

Använd **Spring Security** för att tillämpa rollbaserad åtkomstkontroll (Role-Based Access Control) på era endpoints i de fristående tjänsterna.

3. API-design & Prestanda

KORREKT REST-ARKITEKTUR

Alla endpoints ska designas utifrån strikta REST-principer med korrekta HTTP-metoder för CRUD-operationer (GET, POST, PUT, DELETE).

DOKUMENTATION

Alla API:er ska vara fullständigt dokumenterade med **Swagger/OpenAPI** för enkel testning och integration.

PRESTANDAOPTIMERING

Implementera **caching** på vällästa endpoints för att minska databasbelastningen och optimera prestandan.

4. Testning & 5. CI/CD

HELTÄCKANDE TESTNING

Skriv integrationstester för *minst en* av era microservices. Testerna ska använda **TestContainers** och **MockMvc**.

AGILT SAMARBETE

Arbetet ska ske i ett gemensamt GitHub-repository med ett tydligt agilt arbetsflöde som bygger på **Pull Requests** och **kodgranskning (Code Reviews)**.

AUTOMATISERING (CI/CD)

Sätt upp en **GitHub Actions-pipeline**. Pipelinen ska automatiskt bygga koden, köra testerna och verifiera bygget vid varje Push/PR.

Inlämning & Redovisning

KRAV FÖR INLÄMNING

- **Kod:** Länk till teamets gemensamma GitHub-repository.
- **Dokumentation:** En README.md med en tydlig arkitekturskiss samt instruktioner för att starta systemet lokalt.

MUNTIG REDOVISNING

Projektet ska demonstreras live under sista kursveckan. Redovisningen ska inkludera en genomgång av arkitekturen och er CI/CD-pipeline.

Notera angående bedömning:

Enligt kursplanen är VG-kriteriet specifikt knutet till Individuell Labb 1 och 2. Detta grupprojeckt bedöms därmed enbart med **Godkänt (G)** eller **Ikke Godkänt (IG)**.